

DAA Lab: Celebrity Problem

1. Problem Statement & Objective

Among n people, a **celebrity** is a person who is known by everyone else but knows nobody. The acquaintance matrix is given: $M[i][j] = 1$ means person i knows person j . **Objective:** find the celebrity (if one exists) using brute force and stack-based elimination, and compare them.

2. Approach / Working

- **Brute Force:** check every person against every other person. Person i is the celebrity if $M[i][j] = 0$ and $M[j][i] = 1$ for all $j \neq i$. Time $O(n^2)$, auxiliary space $O(1)$.
- **Stack-Based Elimination:** push all people on a stack. Pop two candidates A and B . If A knows B , A cannot be the celebrity (eliminate A); otherwise B cannot be (eliminate B). Push the survivor back and repeat until one candidate remains, then verify that everyone knows the candidate and the candidate knows nobody. Time $O(n)$, auxiliary space $O(n)$.

3. Algorithm

Brute Force

1. For each person i from 0 to $n-1$, assume i is the celebrity.
2. For every $j \neq i$: if $M[i][j] = 1$ or $M[j][i] = 0$, reject i .
3. If i is not rejected, print i ; if no person qualifies, print "No celebrity".

4. Code

```
#include <stdio.h>

int main()
{
    int n;
    int M[20][20];
    int celebrity = -1;

    printf("Enter number of people: ");
    scanf("%d", &n);

    printf("Enter the acquaintance matrix:\n");
    printf("(1 = knows, 0 = does not know)\n");

    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &M[i][j]);
        }
    }

    // Check each person as a possible celebrity
    for (int i = 0; i < n; i++)
    {
        int isCelebrity = 1;

        for (int j = 0; j < n; j++)
        {
            if (i == j)
                continue;

            // Celebrity knows nobody
            // and everybody must know the celebrity
            if (M[i][j] == 1 || M[j][i] == 0)
            {
                isCelebrity = 0;
                break;
            }
        }

        if (isCelebrity)
        {
            celebrity = i;
            break;
        }
    }

    if (celebrity == -1)
        printf("No celebrity found.\n");
    else
        printf("Celebrity is person %d.\n", celebrity);

    return 0;
}
```

6. Complexity Comparison

Approach	Time	Auxiliary Space
Brute Force	$O(n^2)$	$O(1)$
Stack-Based Elimination	$O(n)$	$O(n)$

7. Conclusion

The stack-based elimination approach is more efficient because every comparison removes exactly one candidate, so only $n-1$ comparisons are needed to reach a single candidate. That candidate is then verified in one $O(n)$ pass, giving $O(n)$ total time versus $O(n^2)$ for brute force, at the cost of $O(n)$ extra space for the stack.